

QResAudit

HFSS Output Extraction and Validation

Complete Implementation Specification for Version 0.1

Prepared for Ahmadh Hassan
23 July 2026

Document status	Build-ready technical specification
Primary scope	HFSS inspection, export, conversion, and offline validation
Target release	qresaudit 0.1.0
Reference stack	Python, PyAEDT, Pydantic, scikit-rf, h5py, Typer, pytest

Purpose: enable a researcher to export a solved HFSS design into a portable, auditable, solver-independent bundle that can be validated and analyzed without AEDT.

Technical note

PyAEDT and AEDT APIs can change between releases. The exporter records exact versions and isolates vendor-specific calls behind adapters. Verify the API compatibility table against the installed version before release.

Document control

Field	Value
Specification version	1.0
Date	23 July 2026
Product	QResAudit
Release covered	0.1.0
Authoring basis	Official PyAEDT and HFSS documentation; current Argonne resonator repository workflow
Intended audience	Primary developer, research collaborators, reviewers, and future maintainers
Implementation language	Python 3.11+
Primary operating systems	Windows for HFSS exporter; Windows, Linux, and macOS for offline core

Definition of complete: Version 0.1 is complete only when the same command can export one driven HFSS fixture and one eigenmode fixture, produce canonical bundles, and pass all offline validation checks without manual modification of the exported files.

Contents

- 1. Product objective and definition of done
- 2. Scope, non-goals, and design principles
- 3. System architecture and trust boundaries
- 4. Repository, packaging, and development environment
- 5. Canonical HFSS export bundle
- 6. Schemas and data contracts
- 7. HFSS inspection and preflight
- 8. Driven Modal and Driven Terminal extraction
- 9. Eigenmode extraction
- 10. Field export, parsing, and HDF5 conversion
- 11. Convergence, mesh, profile, and provenance
- 12. Offline validation engine
- 13. Command-line interface and operational behavior
- 14. Reference implementation skeleton
- 15. Testing and fixtures
- 16. Continuous integration and release engineering
- 17. Milestone backlog and issue plan
- 18. Risks, decisions, and extension points
- 19. Release checklist
- Appendices: complete examples, API map, and sources

1. Product objective and definition of done

QResAudit 0.1 is an extraction and validation layer for solved Ansys HFSS projects. It is not an electromagnetic solver. It converts vendor-specific output into a documented directory whose numerical contents, units, provenance, normalization, and integrity can be checked independently of HFSS.

The immediate problem is not that HFSS cannot export data. The problem is that simulation evidence is usually fragmented across an AEDT project, report definitions, solver result directories, screenshots, handwritten notes, and downstream scripts. A result can be numerically correct yet impossible to reproduce because the setup, variation, port normalization, field scaling, or mesh state was not preserved.

1.1 Primary user story

User story: As a microwave or quantum-device researcher, I can point the exporter at a solved HFSS design and receive a portable bundle containing the numerical outputs and metadata required for another researcher to validate and analyze the result without opening AEDT.

1.2 Version 0.1 acceptance outcome

- [] The exporter opens an existing `.aedt` project without modifying the design.
- [] The `inspect` command lists designs, solution types, setups, sweeps, variations, reports, ports, and solved-data availability.
- [] The `export` command supports one driven solution and one eigenmode solution.
- [] Driven export includes Touchstone, S-parameter report data, convergence, mesh statistics, profile data, variables, and at least one complex H-field grid.
- [] Eigenmode export includes mode frequency/Q data, convergence, mesh statistics, profile data, variables, and complex E/H field grids for selected modes.
- [] Every output file is represented in `manifest.json` with a SHA-256 checksum.
- [] The core validator operates without AEDT, PyAEDT, or an Ansys license.
- [] The validator rejects incomplete, dimensionally inconsistent, mislabeled, or checksum-invalid bundles.
- [] Automated unit and offline integration tests run on a standard public CI runner.
- [] Licensed HFSS integration tests can run on a private Windows runner.

1.3 Quality threshold

Area	Required threshold
Reproducibility	Two independent machines can parse and validate the same bundle with identical normalized metadata.
Determinism	Repeated exports of unchanged solved data produce the same logical manifest and numerically equivalent files; volatile timestamps are excluded from content comparisons.
Traceability	Every derived file records its source raw file and conversion software version.
Failure behavior	Any failed required export causes a non-zero exit code and an invalid/incomplete bundle status.
Numerical fidelity	Touchstone and report round-trips remain within explicit tolerances; no silent unit conversion.
Licensing boundary	No proprietary AEDT result directory or undocumented internal file is required by the offline package.

2. Scope, non-goals, and design principles

2.1 Included in 0.1

- HFSS 3D designs using Driven Modal, Driven Terminal, or Eigenmode solution types.
- Project/design inspection through the documented PyAEDT application interface.
- Nominal and explicitly selected parameter variations.
- Touchstone export for driven solutions.
- Existing report export plus direct solution-data retrieval where report creation is unnecessary.
- Convergence, mesh statistics, solver profile, variable metadata, and mesh visualization export.
- Complex vector E and H fields on Cartesian grids or user-supplied sample points.
- Conversion of raw field exports into HDF5.
- Schema validation, checksums, dimensional checks, normalization checks, and structured diagnostics.

2.2 Explicit non-goals for 0.1

- Creating or modifying HFSS geometries, boundaries, ports, materials, meshes, or solution setups.
- Running an unsolved project automatically unless a later `--solve` extension is approved.
- Resonator Q fitting, cable-delay removal, participation ratios, one-photon normalization, spin coupling, or cooperativity calculations.
- Parsing the complete `.aedt` project format or private `.aedtresults` contents.
- General COMSOL, CST, Palace, or Elmer support.
- HFSS 3D Layout, SBR+, transient, antenna far-field, thermal, or multiphysics extraction.
- Guaranteeing physical correctness of the underlying model. The tool audits evidence, not geometry intent.

2.3 Design principles

Principle	Operational consequence
Exporter/core separation	Only `qresaudit-hfss` imports PyAEDT. `qresaudit-core` remains installable on machines without Ansys.
Raw before derived	Preserve the exact raw vendor export before converting it to normalized CSV or HDF5.
No silent assumptions	Unknown units, missing normalization, missing variation values, or ambiguous solution names become warnings or errors.
Schema first	Files are not considered valid because they exist. They are valid only after schema and semantic validation.
Atomic output	Export into a temporary directory, validate it, then rename it to the requested final path.
Versioned contracts	Every manifest and configuration contains a semantic schema version.
Extensible adapters	Vendor-specific extraction is behind interfaces that can later support Palace or COMSOL.
Research auditability	Warnings, source files, transformations, and software versions are retained in the bundle.

3. System architecture and trust boundaries

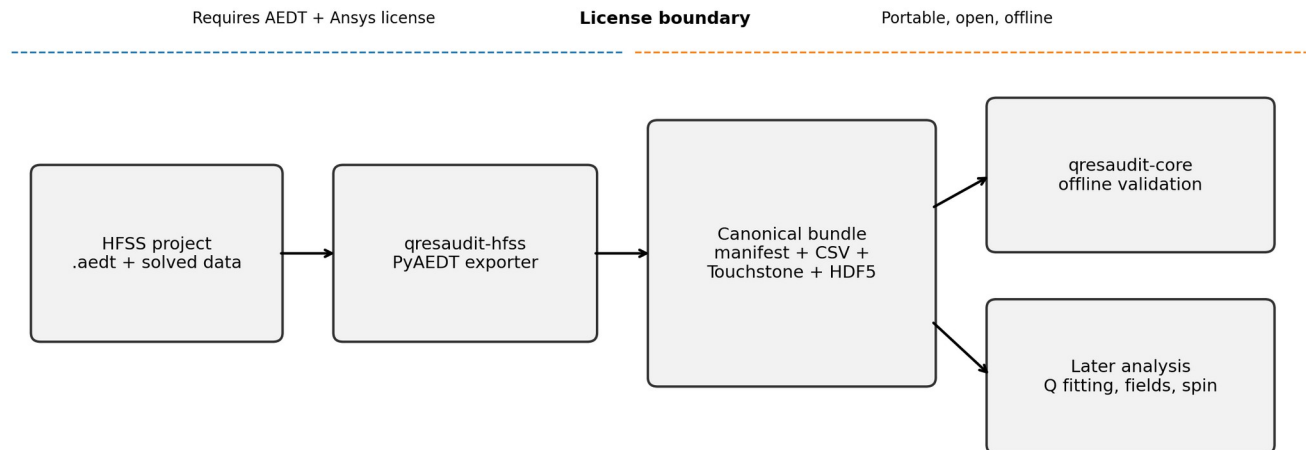


Figure 1. QResAudit separates the licensed HFSS extraction process from portable offline validation and analysis.

3.1 Components

Component	Responsibility	May import PyAEDT
qresaudit-core	Schemas, units, file models, HDF5/CSV readers, checksums, semantic validation, diagnostics, and later analysis.	No
qresaudit-hfss	Open AEDT, inspect solved designs, call PyAEDT exports, capture provenance, convert raw exports, and assemble bundles.	Yes
qresaudit-cli	Typet command groups; routes offline and HFSS commands without importing PyAEDT unnecessarily.	Conditional
qresaudit-testdata	Synthetic bundles and reduced public fixtures.	No

3.2 Transaction lifecycle

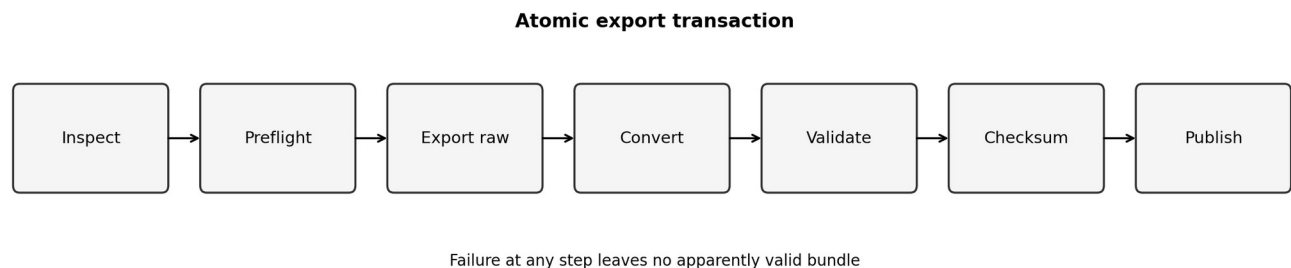


Figure 2. The exporter uses an atomic workflow so a partially written directory cannot be mistaken for a valid bundle.

1. Inspect project and select a single design, setup, sweep, and variation.
2. Preflight required solved data and configuration compatibility.
3. Create a temporary output directory ending in ``.partial``.
4. Export raw vendor files and record each operation.

5. Convert raw files into canonical CSV/HDF5 formats.
6. Build and validate `manifest.json`.
7. Generate SHA-256 checksums for all files except the checksum file itself.
8. Run offline semantic validation against the temporary bundle.
9. Rename the temporary directory to the final destination only if validation passes.

3.3 Trust model

QResAudit treats HFSS as the authority for exported numerical data, but it does not trust file labels or user-provided configuration blindly. It cross-checks the design, solution type, setup, variation, port count, frequency axis, field shape, and unit metadata. Derived values such as field magnitude are recalculated from complex components rather than trusted if present in the raw file.

4. Repository, packaging, and development environment

4.1 Repository structure

```

gresaudit/
├── pyproject.toml
├── README.md
├── LICENSE
├── CHANGELOG.md
├── CONTRIBUTING.md
├── SECURITY.md
├── src/
│   ├── gresaudit/
│   │   ├── __init__.py
│   │   ├── cli.py
│   │   └── models/
│   │       ├── common.py
│   │       ├── config.py
│   │       └── manifest.py
│   │   ├── io/
│   │   │   ├── bundle.py
│   │   │   ├── csv.py
│   │   │   ├── fields_hdf5.py
│   │   │   └── touchstone.py
│   │   ├── validation/
│   │   │   ├── engine.py
│   │   │   ├── checks.py
│   │   │   └── diagnostics.py
│   │   ├── units.py
│   │   └── hashing.py
│   └── gresaudit_hfss/
│       ├── __init__.py
│       ├── cli.py
│       ├── session.py
│       ├── inspect.py
│       ├── exporter.py
│       ├── provenance.py
│       ├── adapters/
│       │   ├── base.py
│       │   ├── driven.py
│       │   └── eigenmode.py
│       ├── exports/
│       │   ├── touchstone.py
│       │   ├── reports.py
│       │   ├── fields.py
│       │   ├── convergence.py
│       │   └── mesh.py
│       └── raw_parsers/
│           └── field_tab.py
├── schemas/
│   ├── manifest-0.1.schema.json
│   └── export-config-0.1.schema.json
├── examples/
│   ├── driven_export.yaml
│   └── eigenmode_export.yaml
├── tests/
│   ├── unit/
│   ├── offline_integration/
│   └── hfss_integration/
├── testdata/
│   └── synthetic/

```

```

├── driven_fixture/
├── eigenmode_fixture/
├── docs/
│   ├── specification.md
│   ├── bundle-format.md
│   └── developer-guide.md

```

4.2 Packaging model

Illustrative `pyproject.toml` dependency block. Pin exact versions in the lock file after testing with the installed AEDT release.

```

[project]
name = "qresaudit"
version = "0.1.0"
requires-python = ">=3.11"
dependencies = [
    "pydantic>=2.13,<3",
    "numpy>=2.0",
    "pandas>=2.2",
    "h5py>=3.16",
    "scikit-rf>=1.8",
    "typer>=0.16",
    "rich>=14",
    "PyYAML>=6.0",
    "platformdirs>=4.0",
]

[project.optional-dependencies]
hfss = ["pyaedt>=1.3,<2"]
dev = [
    "pytest>=8",
    "pytest-cov>=6",
    "hypothesis>=6",
    "ruff>=0.12",
    "mypy>=1.16",
    "pre-commit>=4",
]

[project.scripts]
qresaudit = "qresaudit.cli:app"
qresaudit-hfss = "qresaudit_hfss.cli:app"

```

4.3 Environment rules

- Use a lock file generated from a known working environment. Commit the lock file.
- Test the HFSS exporter only with the supported AEDT/PyAEDT matrix documented in the release notes.
- Do not import `ansys.aedt.core` from `qresaudit-core` modules.
- Use `pathlib.Path` for all paths and normalize Windows paths only at serialization boundaries.
- Use UTF-8 for JSON, YAML, logs, and normalized CSV files.
- Use SI values internally. Preserve original units separately in source metadata.
- Treat warnings as structured objects, not free-form console text alone.

4.4 Recommended licensing

Use a permissive open-source license such as BSD-3-Clause or Apache-2.0. Do not copy Ansys documentation, source code, internal project data, or proprietary result files into the repository. The project should contain adapters that call documented APIs and public synthetic or user-created fixtures only.

5. Canonical HFSS export bundle

5.1 Directory contract

```

hfss_run/
├── manifest.json
├── export_config.resolved.yaml
├── design_variables.json
├── project_variables.json
└── network/

```

```

├── network.s2p
├── network_metadata.json
├── modes/
│   └── eigenmodes.csv
├── convergence/
│   ├── convergence_raw.prof
│   ├── adaptive_passes.csv
│   ├── mesh_stats_raw.txt
│   ├── mesh_summary.json
│   └── solver_profile.prof
├── reports/
│   ├── s_parameters.csv
│   └── report_index.json
├── fields/
│   ├── field_index.json
│   ├── raw/
│   │   ├── mode_01_E.fld
│   │   └── mode_01_H.fld
│   └── hdf5/
│       ├── mode_01_E.h5
│       └── mode_01_H.h5
├── mesh/
│   └── mesh.aedtplt
├── logs/
│   ├── export.jsonl
│   └── export.log
└── checksums.sha256

```

5.2 Required and conditional files

File	Driven	Eigenmode	Requirement
manifest.json	Required	Required	Authoritative bundle index and provenance.
export_config.resolved.yaml	Required	Required	Exact resolved configuration after defaults.
design_variables.json	Required	Required	Evaluated and expression forms where available.
network/network.sNp	Required	Not applicable	Driven S-parameter network.
modes/eigenmodes.csv	Not applicable	Required	Mode frequency and HFSS unloaded-Q evidence.
convergence/*	Required	Required	At least raw convergence and mesh statistics.
reports/s_parameters.csv	Required	Not applicable	Canonical complex report data or equivalent direct solution data.
fields/hdf5/*.h5	At least one	E and H for selected modes	Complex vector fields and metadata.
mesh/mesh.aedtplt	Optional in 0.1	Optional in 0.1	Useful for visual provenance; not parsed in 0.1.
checksums.sha256	Required	Required	Integrity of every bundle file except itself.

5.3 File naming rules

- Use lower-case ASCII names, underscores, and zero-padded mode numbers.
- Do not embed unescaped variable expressions in filenames. Use a deterministic short variation ID.
- A field filename contains mode or frequency context, quantity, and region: `mode\_01\_H\_spin\_sample.h5`.
- A bundle directory may include a human-readable prefix followed by a run ID: `spiral\_setup1\_nominal\_8f2c1d9a`.
- The run ID is the first eight hexadecimal characters of a hash over stable project/design/setup/variation/config metadata, not the file contents.

5.4 Bundle state

State	Meaning
building	Temporary state while export is in progress. Never published.

complete	All required exports succeeded and offline validation passed.
complete_with_warnings	Validation passed but non-fatal warnings remain.
failed	Export or validation failed. Retain only if `--keep-failed` is used.

6. Schemas and data contracts

6.1 Common enumerations

```

from enum import StrEnum

class SolutionKind(StrEnum):
    DRIVEN_MODAL = "driven_modal"
    DRIVEN_TERMINAL = "driven_terminal"
    EIGENMODE = "eigenmode"

class Severity(StrEnum):
    INFO = "info"
    WARNING = "warning"
    ERROR = "error"

class NormalizationKind(StrEnum):
    DRIVEN_EXCITATION_DEPENDENT = "driven_excitation_dependent"
    HFSS_EIGENMODE_PEAK_1 = "hfss_eigenmode_peak_1"
    USER_SCALED = "user_scaled"
    UNKNOWN = "unknown"

class ExportStatus(StrEnum):
    BUILDING = "building"
    COMPLETE = "complete"
    COMPLETE_WITH_WARNINGS = "complete_with_warnings"
    FAILED = "failed"

```

6.2 Manifest schema

```

from datetime import datetime
from pathlib import Path
from pydantic import BaseModel, ConfigDict, Field

class FileRecord(BaseModel):
    model_config = ConfigDict(extra="forbid")
    path: str
    role: str
    media_type: str
    sha256: str
    size_bytes: int = Field(ge=0)
    required: bool
    source_path: str | None = None
    generated_by: str | None = None

class SoftwareRecord(BaseModel):
    name: str
    version: str

class VariationValue(BaseModel):
    expression: str
    evaluated_value: float | None = None
    evaluated_unit: str | None = None

class TouchstoneRecord(BaseModel):
    path: str
    number_of_ports: int = Field(ge=1)
    frequency_unit: str
    parameter_type: str = "S"
    data_format: str
    renormalized: bool
    reference_impedance_ohm: float | None = None
    port_names: list[str]
    frequency_min_hz: float
    frequency_max_hz: float
    point_count: int = Field(ge=1)

class EigenmodeRecord(BaseModel):
    path: str
    mode_count: int = Field(ge=1)
    frequency_unit: str = "Hz"
    q_definition: str = "hfss_unloaded_material_and_boundary_loss"

class FieldRecord(BaseModel):
    path: str
    raw_path: str
    quantity: str
    complex_data: bool
    vector: bool
    units: str
    coordinate_units: str
    coordinate_system: str
    grid_type: str
    region_name: str
    assignment: list[str]
    object_type: str
    solution: str
    mode: int | None = None
    frequency_hz: float | None = None
    phase_deg: float | None = None
    normalization: NormalizationKind
    shape: list[int]
    point_count: int = Field(ge=1)

class Diagnostic(BaseModel):
    code: str
    severity: Severity
    message: str
    path: str | None = None
    context: dict[str, str | int | float | bool] = {}

class HFSSRunManifest(BaseModel):
    model_config = ConfigDict(extra="forbid")
    schema_version: str = "0.1.0"
    exporter_version: str

```

```

bundle_status: ExportStatus
run_id: str
export_timestamp_utc: datetime

project_name: str
project_file_name: str
project_file_sha256: str | None
design_name: str
design_type: str
solution_kind: SolutionKind

setup_name: str
sweep_name: str | None
solution_reference: str
variation_id: str
variation: dict[str, VariationValue]

aedt_version: str
pyaedt_version: str
python_version: str
operating_system: str
model_units: str
reference_coordinate_system: str

ports: list[str]
touchstone: TouchstoneRecord | None
eigenmode: EigenmodeRecord | None
fields: list[FieldRecord]
files: list[FileRecord]
diagnostics: list[Diagnostic]

```

6.3 Export configuration schema

```

class ProjectConfig(BaseModel):
    path: Path
    design: str
    aedt_version: str | None = None
    non_graphical: bool = True
    student_version: bool = False
    remove_lock: bool = False

class SolutionConfig(BaseModel):
    setup: str
    sweep: str | None = None
    variation: dict[str, str] = {}
    modes: list[int] = [1]

class TouchstoneConfig(BaseModel):
    enabled: bool = True
    renormalize: bool = False
    impedance_ohm: float | None = None
    include_gamma_impedance_comments: bool = True

class FieldGridConfig(BaseModel):
    type: str = "Cartesian"
    start: list[str] | None = None
    stop: list[str] | None = None
    step: list[str] | None = None
    center: list[str] | None = None
    sample_points_file: Path | None = None

class FieldExportConfig(BaseModel):
    name: str
    quantity: str
    vector: bool = True
    assignment: list[str] = ["AllObjects"]
    object_type: str = "Vol"
    reference_coordinate_system: str = "Global"
    export_in_si_system: bool = True
    export_field_in_reference: bool = True
    grid: FieldGridConfig
    phase_deg: float | None = 0.0

class ExportConfig(BaseModel):

```

```

schema_version: str = "0.1.0"
project: ProjectConfig
solution: SolutionConfig
touchstone: TouchstoneConfig = TouchstoneConfig()
fields: list[FieldExportConfig] = []
export_existing_reports: bool = True
report_names: list[str] = []
export_convergence: bool = True
export_mesh_stats: bool = True
export_profile: bool = True
export_mesh_visualization: bool = True
keep_raw_exports: bool = True
strict: bool = True

```

Schema rule: Use `extra="forbid"` on externally supplied models. A misspelled key must fail validation rather than being ignored.

6.4 Unit contract

Quantity	Canonical storage	Required source metadata
Frequency	float64 Hz	Original axis unit and conversion factor.
Coordinates	float64 m	Original model/export unit and coordinate system.
Electric field	complex128 V/m	Quantity expression, solution, phase, normalization.
Magnetic field H	complex128 A/m	Quantity expression, solution, phase, normalization.
Magnetic flux density B	complex128 T	Whether exported directly or derived from material relation.
Impedance	float64 ohm	Renormalization status and reference impedance.
Q factor	float64 dimensionless	Definition, extraction source, and mode/solution context.

6.5 Diagnostic code convention

```

HFSS_PROJECT_OPEN_FAILED
HFSS_DESIGN_NOT_FOUND
HFSS_SETUP_NOT_FOUND
HFSS_SWEEP_NOT_FOUND
HFSS_VARIATION_NOT_SOLVED
HFSS_UNSUPPORTED_SOLUTION_TYPE
EXPORT_TOUCHSTONE_FAILED
EXPORT_REPORT_FAILED
EXPORT_FIELD_FAILED
PARSE_FIELD_SHAPE_MISMATCH
VALIDATION_CHECKSUM_MISMATCH
VALIDATION_UNKNOWN_UNIT
VALIDATION_TOUCHSTONE_PORT_MISMATCH
VALIDATION_FIELD_NORMALIZATION_MISSING
VALIDATION_REQUIRED_FILE_MISSING

```

7. HFSS inspection and preflight

7.1 Session creation

```

from ansys.aedt.core import Hfss

app = Hfss(
    project=str(config.project.path),
    design=config.project.design,
    version=config.project.aedt_version,
    non_graphical=config.project.non_graphical,

```

```

new_desktop=False,
close_on_exit=True,
student_version=config.project.student_version,
remove_lock=config.project.remove_lock,
)

```

Open existing projects in read-oriented mode. The exporter must not call methods that alter geometry, solution settings, sources, or reports unless report creation is explicitly required and the project is opened from a temporary copy. The safest default is to export existing reports or retrieve solution data directly.

7.2 Inspect command output

```

$ qresaudit-hfss inspect spiral_resonator.aedt
Project: spiral_resonator
AEDT: 2026.1
Designs:
- SpiralResonator
  solution_type: Eigenmode
  model_units: um
  setups:
    - Setup1
      solved: true
      solutions:
        - Setup1 : LastAdaptive
      modes_available: 5
  reports:
    - Eigenmode Frequency
    - Convergence
  variables:
    trace_width = 2um
    gap = 2um

```

7.3 Machine-readable inspection model

```

class DesignInspection(BaseModel):
    project_name: str
    design_name: str
    design_type: str
    solution_type_raw: str
    solution_kind: SolutionKind | None
    model_units: str
    setups: list[str]
    setup_sweeps: list[str]
    existing_analysis_sweeps: list[str]
    available_variations: list[dict[str, str]]
    reports: list[str]
    ports: list[str]
    excitations: list[str]
    project_variables: dict[str, str]
    design_variables: dict[str, str]
    warnings: list[Diagnostic]

```

7.4 Preflight checks

- [] Project path exists and has `.aedt` or supported archive extension.
- [] Requested design exists and is an HFSS 3D design.
- [] Solution type maps to Driven Modal, Driven Terminal, or Eigenmode.
- [] Requested setup exists.
- [] Driven sweep exists when a sweep is required.
- [] The exact variation is solved or nominal values resolve unambiguously.
- [] Touchstone export is disabled for eigenmode.
- [] Mode numbers are positive and available for eigenmode.
- [] Field grids define either sample points or a valid grid, not both.
- [] Grid steps are positive and implied point count is below configurable safety limits.

[] Assignments exist in the design.

[] Output directory does not overwrite an existing complete bundle unless `--force` is used.

7.5 Variation identity

Never treat an empty variation string as sufficient provenance. Record the expression and evaluated value for every independent design and project variable relevant to the solved state. Build a canonical variation string by sorting variable names and retaining explicit units.

```
def canonical_variation(values: dict[str, str]) -> str:
    return ";".join(f"{name}={values[name]}" for name in sorted(values))

# Example
# gap=2um;trace_width=2um;$temperature=20mK
```

8. Driven Modal and Driven Terminal extraction

8.1 Required sequence

10. Resolve the solution reference as `SetupName : SweepName`.
11. Record port or terminal names and solution type before export.
12. Export Touchstone using explicit setup, sweep, variation, renormalization, and impedance arguments.
13. Read the exported network with scikit-rf and build network metadata.
14. Retrieve or export complex S-parameter data into canonical CSV.
15. Export convergence, mesh statistics, solver profile, variables, and optional mesh visualization.
16. Export configured fields at a stated frequency and phase.
17. Validate agreement between Touchstone and canonical report data.

8.2 Touchstone adapter

```
def export_touchstone(app, config, output_file: Path) -> Path:
    variable_names = list(config.solution.variation)
    variable_values = [config.solution.variation[k] for k in variable_names]

    result = app.export_touchstone(
        setup=config.solution.setup,
        sweep=config.solution.sweep,
        output_file=str(output_file),
        variations=variable_names or None,
        variations_value=variable_values or None,
        renormalization=config.touchstone.renormalize,
        impedance=config.touchstone.impedance_ohm,
        gamma_impedance_comments=(
            config.touchstone.include_gamma_impedance_comments
        ),
    )
    if not result:
        raise ExportError("EXPORT_TOUCHSTONE_FAILED")
    return Path(result)
```

The current documented PyAEDT method accepts setup, sweep, output file, variation names and values, a renormalization flag, a real impedance, and optional Gamma/impedance comments. Record all of these values in the manifest even when defaults are used.

8.3 Canonical S-parameter CSV

```
frequency_hz,
re_S1_1,im_S1_1,
re_S2_1,im_S2_1,
re_S1_2,im_S1_2,
re_S2_2,im_S2_2
```

Store complex components rather than only magnitude or decibels. Magnitude, phase, and decibel columns may be generated as convenience outputs but are not authoritative because they are derivable from the complex data.

8.4 Touchstone validation

- [] Filename extension port count agrees with parsed network port count.
- [] Manifest port count agrees with Touchstone port count.
- [] Frequency values are finite, positive, and strictly increasing after duplicate handling.
- [] All complex matrix values are finite.
- [] Reference impedances are present and compatible with the recorded renormalization state.
- [] Canonical CSV and Touchstone complex data agree within absolute and relative tolerances.
- [] Reciprocity is reported as a diagnostic, not enforced universally.
- [] Passivity is reported as a diagnostic, not enforced for every possible active or numerically imperfect network.

8.5 Direct solution-data extraction

```
expressions = ["S(1,1)", "S(2,1)", "S(1,2)", "S(2,2)"]
data = app.post.get_solution_data(
    expressions=expressions,
    setup_sweep_name=f"{setup} : {sweep}",
    domain="Sweep",
    variations=variations_for_pyaedt,
    primary_sweep_variable="Freq",
)
if not data:
    raise ExportError("EXPORT_SOLUTION_DATA_FAILED")
```

Implementation choice: Prefer direct `SolutionData` retrieval for canonical arrays when stable for the supported PyAEDT version. Retain report CSV export as a fallback and as a way to preserve user-defined reports.

9. Eigenmode extraction

9.1 Semantic restrictions

HFSS eigenmode designs do not use excitations or frequency sweeps. The HFSS-reported unloaded Q represents losses from lossy materials and boundary conditions; it excludes source or port coupling losses because ports and other sources are restricted in eigenmode problems. Label the result accordingly and never rename it to loaded Q, internal Q, or coupling Q without a separate derivation.

9.2 Eigenmode CSV contract

```
mode,
frequency_real_hz,
frequency_imag_hz,
q_hfss_unloaded,
source_solution,
variation_id
```

Some HFSS outputs may expose frequency and Q through reports rather than a single direct API. The adapter should discover available eigenmode report quantities and use a version-specific strategy. The normalized file contract remains fixed even if the vendor extraction path changes.

9.3 Extraction sequence

18. Resolve `SetupName : LastAdaptive` as the eigenmode solution reference.
19. Determine the solved mode count.
20. Extract mode frequencies and HFSS unloaded Q into `modes/eigenmodes.csv`.
21. Export convergence, mesh statistics, profile, variables, and optional mesh visualization.
22. For each selected mode, export complex E and H fields.
23. Mark every raw eigenmode field as `hfss\_eigenmode\_peak\_1` unless source scaling is explicitly detected and recorded.

24. Validate frequency/mode identity across reports and field metadata.

9.4 Mode selection rules

- Configuration mode numbering is one-based to match the HFSS user interface.
- The exporter rejects duplicate or non-positive mode numbers.
- If the selected mode exceeds the solved count, the exporter fails before creating field files.
- The field record includes both requested mode and resolved mode context.
- Mode identity tracking across geometry sweeps is outside 0.1; frequency order alone is not claimed to preserve physical identity.

9.5 Eigenmode normalization

Critical physics rule: HFSS eigenmode field values are normalized to a peak value of 1 because there is no physical excitation amplitude. Raw exported values are relative fields. They must not be interpreted as V/m or A/m per photon until a later energy-normalization module rescales them using a documented stored-energy convention.

10. Field export, parsing, and HDF5 conversion

10.1 Supported v0.1 field quantities

Priority	Quantity	Data type	Typical use
Required	H	Complex vector	Magnetic-field distribution and later spin coupling.
Required	E	Complex vector	Electric concentration and later participation analysis.
Optional	Mag_H / Mag_E	Real scalar	Quick diagnostic only; derive canonically from complex vector.
Deferred	Surface current Jsurf	Complex vector	Conductor loss and current crowding.
Deferred	Energy densities	Real scalar	Stored-energy and participation calculations.
Deferred	Poynting vector	Complex vector	Power-flow diagnostics.

10.2 PyAEDT field export mapping

The documented field calculator exporter can export a named quantity at sample points or on Cartesian, cylindrical, or spherical grids. It supports variation and intrinsic dictionaries, reference coordinate systems, SI-system export, vector/scalar selection, assignments, and surface/volume object types.

```
raw_path = app.post.fields_calculator.export(
    quantity=field.quantity,
    solution=solution_reference,
    variations=variation_dict,
    output_file=str(raw_output),
    intrinsics=intrinsics,
    sample_points=(
        str(field.grid.sample_points_file)
        if field.grid.sample_points_file else None
    ),
    export_with_sample_points=True,
    reference_coordinate_system=field.reference_coordinate_system,
    export_in_si_system=field.export_in_si_system,
    export_field_in_reference=field.export_field_in_reference,
    grid_type=(None if field.grid.sample_points_file else field.grid.type),
    grid_center=field.grid.center,
    grid_start=field.grid.start,
    grid_stop=field.grid.stop,
    grid_step=field.grid.step,
    is_vector=field.vector,
    assignment=field.assignment,
    objects_type=field.object_type,
```


)

10.3 Grid safety calculation

```
def axis_count(start: float, stop: float, step: float) -> int:
    if not (step > 0 and stop >= start):
        raise ValueError("invalid grid axis")
    return int(round((stop - start) / step)) + 1

point_count = nx * ny * nz
if point_count > config.max_field_points:
    raise PreflightError(
        f"field grid requests {point_count:,} points; "
        f"limit is {config.max_field_points:,}"
    )
```

Grid policy: Use coarse global grids and fine local grids. Do not export a nanometer-spaced grid across the entire airbox. Point count must be calculated before HFSS export.

10.4 Raw parser contract

AEDT field exports can differ by quantity, scalar/vector type, real/complex representation, and version. Implement the parser as a tolerant lexical reader plus a strict semantic mapper. Preserve the raw file and include a parser version in the HDF5 metadata.

```
@dataclass(frozen=True)
class ParsedField:
    coordinates_m: np.ndarray          # shape: (N, 3)
    values: np.ndarray                # scalar (N,) or vector (N, 3)
    is_complex: bool
    is_vector: bool
    quantity: str
    value_units: str
    coordinate_units: str
    source_header: list[str]
```

10.5 HDF5 schema

```
/
├── coordinates/
│   ├── points          float64 [N, 3]
│   ├── x               float64 [Nx] # regular grid only
│   ├── y               float64 [Ny]
│   └── z               float64 [Nz]
├── field/
│   ├── real            float64 [N, 3] or [Nx, Ny, Nz, 3]
│   ├── imag            float64 [N, 3] or [Nx, Ny, Nz, 3]
│   └── magnitude        float64 [N] or [Nx, Ny, Nz]
├── metadata/
│   ├── quantity
│   ├── value_units
│   ├── coordinate_units
│   ├── grid_type
│   ├── grid_shape
│   ├── coordinate_system
│   ├── solution_reference
│   ├── setup_name
│   ├── sweep_name
│   ├── mode
│   ├── frequency_hz
│   ├── phase_deg
│   ├── normalization
│   ├── region_name
│   ├── assignments_json
│   ├── source_raw_file
│   ├── source_raw_sha256
│   └── parser_version
```

10.6 HDF5 writer skeleton

```
import h5py
import numpy as np

def write_field_hdf5(path, parsed, metadata):
    values = np.asarray(parsed.values)
    with h5py.File(path, "w") as h5:
        coords = h5.create_group("coordinates")
        coords.create_dataset(
            "points",
            data=np.asarray(parsed.coordinates_m, dtype=np.float64),
            compression="gzip",
            shuffle=True,
        )

        field = h5.create_group("field")
        field.create_dataset(
            "real", data=np.real(values), compression="gzip", shuffle=True
        )
        field.create_dataset(
            "imag", data=np.imag(values), compression="gzip", shuffle=True
        )
        magnitude = (
            np.linalg.norm(values, axis=-1)
            if parsed.is_vector else np.abs(values)
        )
        field.create_dataset(
            "magnitude", data=magnitude, compression="gzip", shuffle=True
        )

        meta = h5.create_group("metadata")
        for key, value in metadata.items():
            meta.attrs[key] = value
```

10.7 Field validation

- [] Coordinate array has shape `(N, 3)` and finite values.
- [] Vector field has shape `(N, 3)` or declared regular-grid equivalent.
- [] Scalar field has one value per coordinate.
- [] Real and imaginary arrays have identical shape.
- [] Magnitude is recomputed and agrees with stored magnitude within tolerance.
- [] Regular-grid coordinate lengths multiply to the declared point count.
- [] Coordinate units and field units are recognized.
- [] Solution, variation, mode/frequency, phase, assignment, and normalization metadata are present.
- [] Source raw checksum agrees with the HDF5 metadata.
- [] No NaN or infinity is accepted unless a future policy explicitly permits masked values.

10.8 Complex-field phase handling

A single real-valued field export at phase 0 degrees is not a complex phasor. Version 0.1 should use a documented complex field quantity or an export path that preserves real and imaginary components. If the installed AEDT version exposes only phase-evaluated real fields for a selected expression, export two orthogonal phases only after verifying the exact convention; otherwise fail with `EXPORT\_COMPLEX\_FIELD\_UNAVAILABLE` rather than manufacturing complex data.

11. Convergence, mesh, profile, and provenance

11.1 Vendor exports

Evidence	PyAEDT method	v0.1 handling
Convergence	Hfss.export_convergence(setup, variations, output_file)	Preserve raw file; parse stable tabular fields when possible.
Mesh statistics	Hfss.export_mesh_stats(setup, variations, output_file)	Preserve raw file; normalize totals and per-object statistics.
Solver profile	Hfss.export_profile(setup, variation, output_file)	Preserve raw PROF file; extract timing/memory only if parser is reliable.
Mesh visualization	post.export_mesh_obj(setup, intrinsics, export_air_objects, on_surfaces)	Preserve AEDTPLT; do not parse tetrahedra in 0.1.

11.2 Normalized adaptive-pass table

```
pass_index,
frequency_hz,
delta_frequency_percent,
maximum_delta_s,
tetrahedra,
converged,
elapsed_time_s,
peak_memory_bytes
```

Not every column exists for every solution type. Use nullable columns and record the parser confidence. Do not infer missing values from screenshots or unrelated reports.

11.3 Convergence policy in 0.1

Check	Severity
Required raw convergence file missing	Error
No parsed pass rows but raw file exists	Warning
Final pass explicitly unconverged	Warning by default; error in strict mode
Frequency changed upward or downward	Informational; sign is not itself a failure
Tetrahedra do not increase monotonically	Warning, not automatic failure
Requested design quantity lacks convergence evidence	Warning in 0.1; later quantity-specific convergence module

11.4 Provenance capture

- [] AEDT version and build identifier where accessible.
- [] PyAEDT version.
- [] Python version and operating system.
- [] Project name, design name, design type, and solution type.
- [] Project file hash if hashing is feasible and permitted.
- [] Setup, sweep, solution reference, variation, modes, frequency, and phase.
- [] Model units and reference coordinate system.
- [] Port/terminal names and reference impedance information.
- [] Design and project variables in expression form and evaluated form where available.
- [] Configuration file after environment variables and defaults are resolved.
- [] Every raw-to-canonical conversion with source and destination hashes.

11.5 Variable export

```
{
  "gap": {
```

```

    "expression": "2um",
    "evaluated_value": 2e-6,
    "evaluated_unit": "m",
    "scope": "design"
  },
  "$temperature": {
    "expression": "20mK",
    "evaluated_value": 0.02,
    "evaluated_unit": "K",
    "scope": "project"
  }
}

```

12. Offline validation engine

12.1 Validation stages

25. Filesystem validation: expected directory, readable files, no path traversal, no duplicate logical paths.
26. Schema validation: parse `manifest.json` with the exact schema version.
27. Integrity validation: verify size and SHA-256 for every file record.
28. Format validation: parse JSON, YAML, CSV, Touchstone, and HDF5.
29. Semantic validation: cross-check ports, shapes, units, solution context, and normalization.
30. Cross-file validation: compare Touchstone and CSV, manifest and field metadata, raw and derived hashes.
31. Policy validation: required outputs by solution type and strictness level.

12.2 Validator interface

```

@dataclass(frozen=True)
class ValidationResult:
    valid: bool
    diagnostics: tuple[Diagnostic, ...]

    @property
    def errors(self):
        return tuple(d for d in self.diagnostics if d.severity == Severity.ERROR)

def validate_bundle(path: Path, strict: bool = True) -> ValidationResult:
    diagnostics: list[Diagnostic] = []
    manifest = load_manifest(path / "manifest.json")
    diagnostics += validate_filesystem(path, manifest)
    diagnostics += validate_checksums(path, manifest)
    diagnostics += validate_solution_contract(path, manifest, strict=strict)
    diagnostics += validate_touchstone(path, manifest)
    diagnostics += validate_fields(path, manifest)
    diagnostics += validate_cross_file_consistency(path, manifest)
    return ValidationResult(
        valid=not any(d.severity == Severity.ERROR for d in diagnostics),
        diagnostics=tuple(diagnostics),
    )

```

12.3 Required semantic checks

Check	Driven	Eigenmode
Manifest solution kind matches required files	Yes	Yes
Touchstone exists and parses	Yes	N/A
Port count consistency	Yes	N/A
Eigenmode table exists and parses	N/A	Yes
Selected modes exist in table	N/A	Yes
At least one field record	Yes	Yes
E and H for every selected mode	Optional in minimal driven fixture	Yes
Normalization metadata	Yes	Yes
Convergence and mesh evidence	Yes	Yes
Checksums	Yes	Yes

12.4 Exit codes

Code	Meaning
0	Operation succeeded; no validation errors.
1	Validation failed or export completed with required-output failure.
2	Configuration or command-line usage error.
3	AEDT/PyAEDT environment or license/session failure.
4	Project/design/setup/variation preflight failure.
5	Unexpected internal error.

12.5 Strict and permissive modes

Strict mode is the release default. Permissive mode exists for development and legacy projects. It may downgrade selected missing optional evidence from error to warning, but it must never ignore checksum mismatches, malformed files, unknown normalization, or contradictory solution metadata.

13. Command-line interface and operational behavior

13.1 Commands

```
qresaudit-hfss inspect PROJECT [--design DESIGN] [--json]
qresaudit-hfss export PROJECT --config CONFIG --output OUTPUT
qresaudit validate BUNDLE [--strict/--no-strict] [--json]
qresaudit show BUNDLE
qresaudit schema manifest
qresaudit schema config
```

13.2 Export command behavior

```
$ qresaudit-hfss export spiral.aedt \
  --config examples/eigenmode_export.yaml \
  --output exports/spiral_mode1

Preflight                PASS
Project opened           spiral.aedt
Design                   SpiralResonator
Solution                 Setup1 : LastAdaptive
Variation                nominal
Eigenmode table          PASS 5 modes
Convergence              PASS
Mesh statistics           PASS
Field mode_01_E          PASS 1,240,000 points
Field mode_01_H          PASS 1,240,000 points
Manifest                 PASS
Checksums                PASS
Offline validation       PASS with 1 warning
Bundle                   exports/spiral_mode1
```

13.3 Structured logging

```
{
  "time": "2026-07-23T21:02:10Z",
  "level": "INFO",
  "event": "export.started",
  "project": "spiral.aedt"
}
{
  "time": "2026-07-23T21:02:18Z",
  "level": "INFO",
  "event": "field.exported",
  "quantity": "H",
  "mode": 1,
  "raw_path": "fields/raw/mode_01_H fld",
  "points": 1240000
}
{
  "time": "2026-07-23T21:02:30Z",
  "level": "WARNING",
  "event": "convergence.parse_partial",
  "code": "CONVERGENCE_UNKNOWN_COLUMN"
}
```

13.4 Operational safeguards

- [] Require `--force` to replace an existing destination.
- [] Never delete a project lock unless `remove\_lock: true` is explicit.
- [] Refuse output paths inside the `.aedresults` directory.
- [] Calculate field point counts before calling AEDT.
- [] Support `--keep-failed` for debugging, otherwise remove `.partial` directories.
- [] Always release AEDT in `finally` or context-manager cleanup.
- [] Redact machine usernames and sensitive absolute paths from public manifests when `portable\_paths: true` is selected.

14. Reference implementation skeleton

14.1 Export adapter protocol

```
from typing import Protocol

class HFSSAdapter(Protocol):
    def preflight(self) -> list[Diagnostic]: ...
    def export_primary_results(self, staging: Path) -> list[FileRecord]: ...
    def export_evidence(self, staging: Path) -> list[FileRecord]: ...
    def export_fields(self, staging: Path) -> tuple[list[FileRecord], list[FieldRecord]]: ...

def adapter_for(app, config) -> HFSSAdapter:
    kind = map_solution_kind(app.solution_type)
    if kind in {SolutionKind.DRIVEN_MODAL, SolutionKind.DRIVEN_TERMINAL}:
        return DrivenAdapter(app, config)
    if kind is SolutionKind.EIGENMODE:
        return EigenmodeAdapter(app, config)
    raise UnsupportedSolutionType(app.solution_type)
```

14.2 Export orchestrator

```
def export_bundle(config: ExportConfig, output: Path) -> Path:
    staging = output.with_name(output.name + ".partial")
    prepare_empty_staging(staging)
    diagnostics: list[Diagnostic] = []

    try:
        with open_hfss_session(config.project) as app:
            inspection = inspect_design(app)
            diagnostics += run_preflight(inspection, config)
            raise_on_errors(diagnostics)

            adapter = adapter_for(app, config)
            files = []
            files += export_variables(app, staging)
            files += adapter.export_primary_results(staging)
            files += adapter.export_evidence(staging)
            field_files, field_records = adapter.export_fields(staging)
            files += field_files

            manifest = build_manifest(
                app=app,
                config=config,
                inspection=inspection,
                files=files,
                fields=field_records,
                diagnostics=diagnostics,
                status=ExportStatus.BUILDING,
            )
            write_manifest(staging, manifest)

            refresh_hashes_and_sizes(staging)
            result = validate_bundle(staging, strict=config.strict)
            if not result.valid:
```

```

        raise BundleValidationError(result)

    finalize_manifest_status(staging, result)
    write_checksums(staging)
    atomic_publish(staging, output)
    return output
except Exception:
    mark_failed_if_possible(staging)
    if not config.keep_failed:
        safe_remove(staging)
    raise

```

14.3 Checksum implementation

```

def sha256_file(path: Path, chunk_size: int = 1024 * 1024) -> str:
    digest = hashlib.sha256()
    with path.open("rb") as stream:
        while chunk := stream.read(chunk_size):
            digest.update(chunk)
    return digest.hexdigest()

def write_checksums(bundle: Path) -> None:
    paths = sorted(
        p for p in bundle.rglob("")
        if p.is_file() and p.name != "checksums.sha256"
    )
    lines = [f"{sha256_file(p)} {p.relative_to(bundle).as_posix()}" for p in paths]
    (bundle / "checksums.sha256").write_text("\n".join(lines) + "\n", encoding="utf-8")

```

14.4 Touchstone reader wrapper

```

import skrf as rf

def load_network(path: Path) -> rf.Network:
    try:
        network = rf.Network(str(path))
    except Exception as exc:
        raise DataFormatError("TOUCHSTONE_PARSE_FAILED") from exc
    if network.nports < 1:
        raise DataFormatError("TOUCHSTONE_NO_PORTS")
    if not np.all(np.isfinite(network.f)):
        raise DataFormatError("TOUCHSTONE_NONFINITE_FREQUENCY")
    if not np.all(np.diff(network.f) > 0):
        raise DataFormatError("TOUCHSTONE_NONMONOTONIC_FREQUENCY")
    if not np.all(np.isfinite(network.s)):
        raise DataFormatError("TOUCHSTONE_NONFINITE_S")
    return network

```

14.5 Diagnostic helpers

```

def error(code: str, message: str, **context) -> Diagnostic:
    return Diagnostic(
        code=code,
        severity=Severity.ERROR,
        message=message,
        context=context,
    )

def warning(code: str, message: str, **context) -> Diagnostic:
    return Diagnostic(
        code=code,
        severity=Severity.WARNING,
        message=message,
        context=context,
    )

```

14.6 Typer CLI skeleton

```
import typer
from pathlib import Path

app = typer.Typer(no_args_is_help=True)

@app.command()
def validate(
    bundle: Path,
    strict: bool = typer.Option(True, "--strict/--no-strict"),
    json_output: bool = typer.Option(False, "--json"),
) -> None:
    result = validate_bundle(bundle, strict=strict)
    render_validation_result(result, json_output=json_output)
    raise typer.Exit(code=0 if result.valid else 1)
```

14.7 API compatibility isolation

```
class PyAEDTCapabilities(BaseModel):
    version: str
    has_fields_calculator_export: bool
    has_export_touchstone: bool
    has_export_convergence: bool
    has_export_mesh_stats: bool
    has_export_profile: bool
    has_export_mesh_obj: bool

def detect_capabilities(app) -> PyAEDTCapabilities:
    return PyAEDTCapabilities(
        version=importlib.metadata.version("pyaedt"),
        has_fields_calculator_export=hasattr(
            app.post.fields_calculator, "export"
        ),
        has_export_touchstone=hasattr(app, "export_touchstone"),
        has_export_convergence=hasattr(app, "export_convergence"),
        has_export_mesh_stats=hasattr(app, "export_mesh_stats"),
        has_export_profile=hasattr(app, "export_profile"),
        has_export_mesh_obj=hasattr(app.post, "export_mesh_obj"),
    )
```

Coding rule: Do not scatter `hasattr` checks throughout the exporter. Detect capabilities once, choose an adapter strategy, and fail with a clear supported-version diagnostic.

15. Testing and fixtures

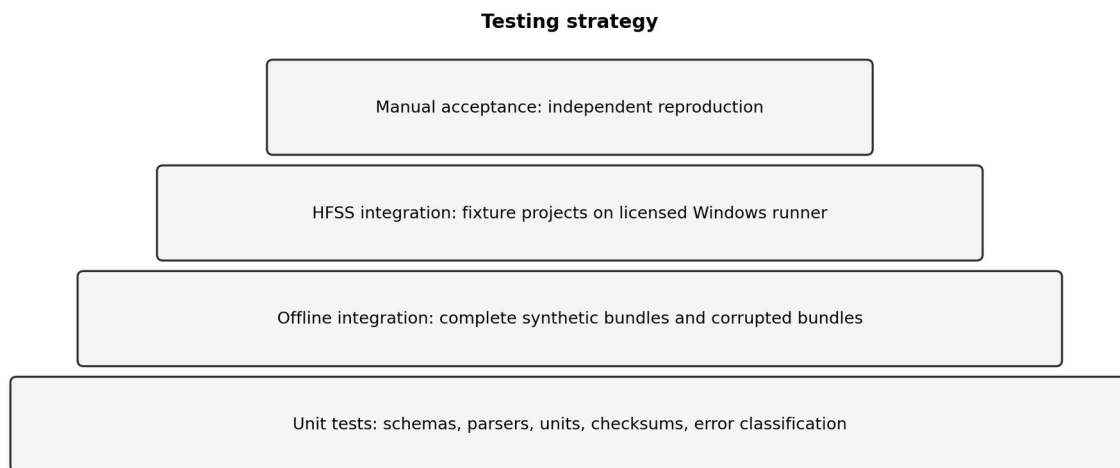


Figure 3. Most tests remain open and license-free; only the narrow vendor boundary requires a licensed runner.

15.1 Unit tests

- Pydantic model acceptance and rejection cases.
- Unit parser and SI conversion.
- Canonical variation ordering and run-ID stability.
- SHA-256 generation and checksum-file parsing.
- Touchstone parser behavior on valid and malformed files.
- Field raw parser on scalar/vector and real/complex synthetic files.
- HDF5 writer/reader round-trip.
- Grid shape and point-count calculations.
- Diagnostic severity and strict-mode promotion rules.

15.2 Property-based tests

- Random valid grid dimensions always generate consistent point counts.
- Random complex fields round-trip through HDF5 within float64 tolerance.
- Manifest serialization/deserialization preserves all fields.
- Any one-byte mutation of a file causes a checksum mismatch.
- Invalid or unknown units never pass silently.

15.3 Offline integration fixtures

Fixture	Purpose
valid_driven_minimal	Small two-port Touchstone, S CSV, convergence, one H field, complete manifest.
valid_eigenmode_minimal	Two modes, E/H fields for mode 1, convergence, complete manifest.
corrupt_checksum	One byte modified after checksum generation.
missing_normalization	Eigenmode field lacks normalization metadata.
port_mismatch	Manifest says two ports while Touchstone contains one.
field_shape_mismatch	Coordinate and value counts differ.
unknown_unit	Field uses an unsupported or misspelled unit.
partial_bundle	Bundle state is building and required file is absent.

15.4 HFSS integration fixtures

Fixture A: driven	Fixture B: eigenmode
-------------------	----------------------

Simple two-port notch or transmission resonator.	Simple cavity or lumped resonator with at least two solved modes.
Small enough for rapid solve and export.	Small enough for rapid solve and field export.
Known setup and sweep names.	Known setup and LastAdaptive solution.
One configured H grid.	Mode-1 E and H grids.
No proprietary paper geometry required.	No proprietary paper geometry required.

15.5 Numerical tolerances

Comparison	Initial tolerance
Touchstone frequency axis	Exact after conversion to Hz, or absolute tolerance $\leq 1e-6$ Hz for text parsing.
Complex S values	rtol 1e-9, atol 1e-12 unless export precision requires documented relaxation.
HDF5 real/imag round-trip	Exact for written float64 arrays.
Derived magnitude	rtol 1e-12, atol 1e-15.
Grid reconstruction	Coordinates agree within half of last printed decimal resolution.

15.6 Manual acceptance test

32. Clone the repository on a clean machine.
33. Install `qresaudit[hfss]` in a new environment.
34. Run `inspect` on both fixture projects.
35. Run `export` using committed example configurations.
36. Copy the resulting bundles to a Linux machine without Ansys.
37. Install base `qresaudit` only.
38. Run `qresaudit validate` and obtain the expected status.
39. Open Touchstone and HDF5 with independent tools and compare representative values against HFSS.

16. Continuous integration and release engineering

16.1 Public CI

```

name: core-ci
on: [push, pull_request]

jobs:
  test:
    strategy:
      matrix:
        os: [ubuntu-latest, windows-latest, macos-latest]
        python: ["3.11", "3.12", "3.13"]
    runs-on: ${ matrix.os }
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: ${ matrix.python }
      - run: pip install -e ".[dev]"
      - run: ruff check .
      - run: mypy src
      - run: pytest tests/unit tests/offline_integration --cov=qresaudit

```

16.2 Private licensed CI

- Self-hosted Windows runner with a supported AEDT installation and valid license.
- Run only on protected branches, scheduled nightly jobs, and release candidates.
- Use dedicated small fixture projects.
- Never upload proprietary `.aedt` files or result directories as public artifacts.
- Upload only normalized test summaries or approved synthetic bundles.
- Pin AEDT and PyAEDT versions in runner metadata.

16.3 Release artifacts

- [] Source distribution and wheel build successfully.
- [] Wheel installs without PyAEDT for base package.
- [] HFSS extra installs on supported Windows environment.
- [] JSON schemas are included in package data.
- [] Example configurations validate.
- [] Offline fixture bundles validate.
- [] Documentation builds.
- [] Changelog lists schema and compatibility changes.
- [] Git tag, package version, manifest exporter version, and documentation version agree.

16.4 Compatibility policy

Change	Version action
Add optional manifest field	Minor package release; same schema if truly optional and backward compatible.
Rename or remove manifest field	New schema version; migration utility required.
Change HDF5 dataset meaning or shape	New field-format version and reader compatibility layer.
Add supported AEDT/PyAEDT pair	Patch or minor release after licensed integration tests.
Drop a Python or AEDT version	Minor release with explicit deprecation notice.

17. Milestone backlog and issue plan

17.1 Milestone M0 - repository foundation

Issue	Deliverable	Acceptance
M0-1	Initialize package, licensing, formatting, typing, test configuration.	Clean install and empty CI pass.
M0-2	Implement diagnostic models and exception hierarchy.	Unit-tested stable diagnostic codes.
M0-3	Implement configuration models and JSON schema export.	Valid examples pass; misspelled keys fail.
M0-4	Implement manifest models and schema export.	Round-trip tests and `extra=forbid`.
M0-5	Implement checksums and atomic staging directory.	Mutation and failed-export tests.

17.2 Milestone M1 - offline core

Issue	Deliverable
M1-1	Bundle loader and path-safety checks.
M1-2	Touchstone reader and canonical network metadata.
M1-3	HDF5 field writer/reader and schema validation.
M1-4	CSV readers for modes, reports, and convergence.
M1-5	Validation engine and human/JSON rendering.
M1-6	Synthetic valid and invalid fixture bundles.

17.3 Milestone M2 - HFSS inspection

Issue	Deliverable
M2-1	PyAEDT session wrapper with guaranteed cleanup.
M2-2	Project/design/setup/sweep inspection.
M2-3	Variation and variable capture.
M2-4	Capability detection and supported-version matrix.

M2-5	Preflight checks and `inspect --json`.

17.4 Milestone M3 - driven exporter

Issue	Deliverable
M3-1	Touchstone export and metadata.
M3-2	Complex S-data extraction and CSV.
M3-3	Convergence, mesh statistics, profile, mesh visualization.
M3-4	First complex H-field export and raw parser.
M3-5	Driven bundle assembly and licensed fixture test.

17.5 Milestone M4 - eigenmode exporter

Issue	Deliverable
M4-1	Mode frequency and HFSS unloaded-Q extraction.
M4-2	Mode selection and solution-context handling.
M4-3	Complex E/H exports with normalization metadata.
M4-4	Eigenmode bundle assembly and licensed fixture test.

17.6 Milestone M5 - release hardening

Issue	Deliverable
M5-1	Cross-platform offline CI and coverage threshold.
M5-2	Private licensed CI workflow.
M5-3	User documentation and complete examples.
M5-4	Independent manual acceptance test.
M5-5	0.1.0 release and archived fixture bundle checksums.

17.7 Recommended implementation order

40. Manifest, configuration, diagnostics, checksums, and bundle staging.
41. Offline HDF5 and Touchstone readers plus validation.
42. HFSS inspection and capability detection.
43. Driven Touchstone and convergence exports.
44. Single H-field export and parser.
45. Complete driven fixture.
46. Eigenmode frequency/Q extraction.
47. Eigenmode E/H field export and normalization metadata.
48. Complete eigenmode fixture.
49. CI, documentation, and release hardening.

18. Risks, decisions, and extension points

18.1 Primary risks

Risk	Mitigation
PyAEDT API changes	Capability detection, version matrix, adapter isolation, licensed CI.
Field export format changes	Retain raw files, parser fixtures by AEDT version, explicit parser version.
Complex field ambiguity	Fail unless complex representation is verified; preserve phase and expression metadata.
Large field files	Preflight point limits, region-specific grids, gzip/shuffle HDF5 compression.
Inconsistent report names	Use direct solution data where possible; treat existing report

	export as an indexed optional source.
Eigenmode normalization misuse	Mandatory normalization enum and validation error when absent.
Project mutation during export	Read-only operational policy; temporary project copy if report creation is unavoidable.
Licensing restrictions on fixtures	Use original minimal geometries and publish normalized outputs only.
False sense of physical correctness	Clearly separate evidence validation from geometry/model verification.

18.2 Architecture decisions

Decision	Reason
JSON manifest + YAML user config	JSON is strict and machine-oriented; YAML is convenient for authored configuration.
HDF5 for fields	Supports large multidimensional arrays, metadata attributes, compression, and portable readers.
CSV for compact tables	Human-inspectable and language-neutral.
Touchstone retained unchanged	Standard network-data interchange and independent validation.
Pydantic v2 models	Runtime validation, serialization, and JSON Schema generation from one contract.
Typer CLI	Typed command interfaces with clear help and exit behavior.
No database in 0.1	Bundles remain portable directories; indexing can be added later.

18.3 Deferred extension points

- One-photon and stored-energy normalization.
- Electric, magnetic, dielectric, surface, and conductor participation ratios.
- Mode tracking by field overlap across parameter sweeps.
- Mesh convergence and asymptotic error estimation.
- Resonator fitting and uncertainty quantification.
- Spin filling factor, single-spin coupling, collective coupling, and cooperativity.
- Palace, COMSOL, CST, and Elmer exporters.
- Bundle indexing, dashboards, and web reports.

18.4 Integration with the existing Argonne repository

Use the existing resonator projects as later real-world regression cases, not as the first public fixtures. The repository already contains multiple scripted HFSS models and documented analytic/extraction workflows. After 0.1 is stable on minimal fixtures, export one bundle for each resonator topology and add a small `qresaudit.yaml` file beside each project. This will convert the current collection into a reproducible benchmark set without coupling the package design to any one geometry.

19. Release checklist

19.1 Functional completion

- ☐ Driven Modal fixture exports successfully.
- ☐ Driven Terminal behavior is either tested or explicitly marked experimental.
- ☐ Eigenmode fixture exports successfully.
- ☐ Touchstone export records variation and renormalization settings.
- ☐ Complex S arrays are available in canonical form.
- ☐ Convergence, mesh statistics, and profile are preserved.
- ☐ Complex H field exports for driven fixture.

- [] Complex E and H fields export for selected eigenmode.
- [] Eigenmode normalization is explicitly labeled.
- [] Manifest and checksum generation are deterministic.
- [] Offline validator passes valid fixtures and rejects corrupt fixtures.

19.2 Engineering completion

- [] Public CI passes on supported Python versions and operating systems.
- [] Licensed integration suite passes on the documented AEDT/PyAEDT pair.
- [] No core module imports PyAEDT.
- [] Test coverage meets the project threshold for core modules.
- [] Static typing and linting pass.
- [] All CLI commands have help text and stable exit codes.
- [] Exceptions are converted into structured diagnostics at the CLI boundary.
- [] Temporary files and AEDT sessions are cleaned up after success and failure.

19.3 Documentation completion

- [] README contains installation, architecture, quick start, and limitations.
- [] Bundle format is documented independently of Python implementation.
- [] Configuration schema and examples are documented.
- [] Supported AEDT/PyAEDT versions are listed.
- [] Normalization and Q terminology are unambiguous.
- [] Contribution, security, and release procedures are present.
- [] A complete driven and eigenmode tutorial is reproducible from a clean environment.

Appendix A. Complete driven export configuration

```

schema_version: "0.1.0"

project:
  path: "./fixtures/driven/notch_resonator.aedt"
  design: "NotchResonator"
  aedt_version: "2026.1"
  non_graphical: true
  student_version: false
  remove_lock: false

solution:
  setup: "Setup1"
  sweep: "Sweep1"
  variation: {}
  modes: []

touchstone:
  enabled: true
  renormalize: false
  impedance_ohm: null
  include_gamma_impedance_comments: true

export_existing_reports: true
report_names:
  - "S Parameters"

export_convergence: true
export_mesh_stats: true
export_profile: true
export_mesh_visualization: true
keep_raw_exports: true
strict: true

```

```

fields:
- name: "device_H"
  quantity: "H"
  vector: true
  assignment: ["Substrate", "Signal", "Ground", "AirBox"]
  object_type: "Vol"
  reference_coordinate_system: "Global"
  export_in_si_system: true
  export_field_in_reference: true
  phase_deg: 0.0
  grid:
    type: "Cartesian"
    start: ["-800um", "-800um", "-20um"]
    stop: ["800um", "800um", "100um"]
    step: ["10um", "10um", "5um"]

```

Appendix B. Complete eigenmode export configuration

```

schema_version: "0.1.0"

project:
  path: "./fixtures/eigenmode/cavity_resonator.aedt"
  design: "CavityResonator"
  aedt_version: "2026.1"
  non_graphical: true
  student_version: false
  remove_lock: false

solution:
  setup: "Setup1"
  sweep: null
  variation: {}
  modes: [1, 2]

touchstone:
  enabled: false
  renormalize: false
  impedance_ohm: null
  include_gamma_impedance_comments: false

export_existing_reports: true
report_names:
- "Eigenmode Frequency"
- "Q Factor"

export_convergence: true
export_mesh_stats: true
export_profile: true
export_mesh_visualization: true
keep_raw_exports: true
strict: true

fields:
- name: "mode_E"
  quantity: "E"
  vector: true
  assignment: ["AllObjects"]
  object_type: "Vol"
  reference_coordinate_system: "Global"
  export_in_si_system: true
  export_field_in_reference: true
  phase_deg: 0.0
  grid:
    type: "Cartesian"
    start: ["-1mm", "-1mm", "-0.2mm"]
    stop: ["1mm", "1mm", "0.5mm"]
    step: ["20um", "20um", "10um"]

- name: "mode_H"
  quantity: "H"
  vector: true
  assignment: ["AllObjects"]
  object_type: "Vol"
  reference_coordinate_system: "Global"
  export_in_si_system: true

```

```

export_field_in_reference: true
phase_deg: 0.0
grid:
  type: "Cartesian"
  start: ["-1mm", "-1mm", "-0.2mm"]
  stop: ["1mm", "1mm", "0.5mm"]
  step: ["20um", "20um", "10um"]

```

Appendix C. Example manifest

```

{
  "schema_version": "0.1.0",
  "exporter_version": "0.1.0",
  "bundle_status": "complete_with_warnings",
  "run_id": "8f2c1d9a",
  "export_timestamp_utc": "2026-07-23T21:02:30Z",
  "project_name": "cavity_resonator",
  "project_file_name": "cavity_resonator.aedt",
  "project_file_sha256": null,
  "design_name": "CavityResonator",
  "design_type": "HFSS",
  "solution_kind": "eigenmode",
  "setup_name": "Setup1",
  "sweep_name": null,
  "solution_reference": "Setup1 : LastAdaptive",
  "variation_id": "nominal",
  "variation": {},
  "aedt_version": "2026.1",
  "pyaedt_version": "1.3.0",
  "python_version": "3.12.4",
  "operating_system": "Windows-11",
  "model_units": "um",
  "reference_coordinate_system": "Global",
  "ports": [],
  "touchstone": null,
  "eigenmode": {
    "path": "modes/eigenmodes.csv",
    "mode_count": 2,
    "frequency_unit": "Hz",
    "q_definition": "hfss_unloaded_material_and_boundary_loss"
  },
  "fields": [
    {
      "path": "fields/hdf5/mode_01_H.h5",
      "raw_path": "fields/raw/mode_01_H.fld",
      "quantity": "H",
      "complex_data": true,
      "vector": true,
      "units": "A/m",
      "coordinate_units": "m",
      "coordinate_system": "Global",
      "grid_type": "Cartesian",
      "region_name": "mode_H",
      "assignment": ["AllObjects"],
      "object_type": "Vol",
      "solution": "Setup1 : LastAdaptive",
      "mode": 1,
      "frequency_hz": 6.173842e9,
      "phase_deg": 0.0,
      "normalization": "hfss_eigenmode_peak_1",
      "shape": [101, 101, 71, 3],
      "point_count": 724271
    }
  ],
  "files": [],
  "diagnostics": [
    {
      "code": "CONVERGENCE_PARSE_PARTIAL",
      "severity": "warning",
      "message": "Raw convergence was preserved but one profile column was not normalized.",
      "path": "convergence/convergence_raw.prof",
      "context": {}
    }
  ]
}

```


Appendix D. API compatibility map

Capability	Current documented PyAEDT interface	Implementation note
Open HFSS	Hfss(project, design, solution_type, setup, version, non_graphical, new_desktop, close_on_exit, ...)	Wrap in context manager and record version.
Touchstone	Hfss.export_touchstone(setup, sweep, output_file, variations, variations_value, renormalization, impedance, gamma_impedance_comments)	Driven only; record explicit defaults.
Convergence	Hfss.export_convergence(setup, variations, output_file)	Preserve raw and parse defensively.
Mesh stats	Hfss.export_mesh_stats(setup, variations, output_file)	Preserve raw and normalize stable fields.
Profile	Hfss.export_profile(setup, variation, output_file)	Raw PROF is authoritative.
Report CSV	PostProcessor3D.export_report_to_csv(project_dir, plot_name, uniform, start, end, step, ...)	Existing reports; avoid mutating project.
Solution data	PostProcessor3D.get_solution_data(expression, setup_sweep_name, domain, variations, primary_sweep_variable, ...)	Preferred canonical numeric extraction where stable.
Field grid	FieldsCalculator.export(quantity, solution, variations, output_file, intrinsics, sample_points, reference_coordinate_system, grid_type, grid_start, grid_stop, grid_step, is_vector, assignment, objects_type, ...)	Raw format parser needs version fixtures.
Mesh visualization	PostProcessor3D.export_mesh_obj(setup, intrinsics, export_air_objects, on_surfaces)	Store AEDTPLT only in 0.1.

Appendix E. Terminology

Term	Meaning in this specification
Bundle	Portable directory containing manifest, raw exports, normalized files, logs, and checksums.
Raw export	File emitted directly by AEDT/PyAEDT before QResAudit conversion.
Canonical file	Normalized CSV, JSON, YAML, Touchstone, or HDF5 governed by this specification.
Variation	Complete set of project/design variable values selecting a solved state.
Solution reference	AEDT context such as `Setup1 : Sweep1` or `Setup1 : LastAdaptive`.
Normalization	Rule defining the scale of a field array.
HFSS unloaded Q	Eigenmode Q due to lossy materials and boundary conditions; excludes source/port coupling losses.
Strict mode	Validation policy in which required evidence failures are errors.

Appendix F. Official sources

- [1] PyAEDT documentation, Hfss class, current stable documentation.
https://aedt.docs.pyansys.com/version/stable/API/_autosummary/ansys.aedt.core.hfss.Hfss.html
- [2] PyAEDT documentation, Hfss.export_touchstone.
https://aedt.docs.pyansys.com/version/stable/API/_autosummary/ansys.aedt.core.hfss.Hfss.export_touchstone.html
- [3] PyAEDT documentation, FieldsCalculator.export.
https://aedt.docs.pyansys.com/version/stable/API/visualization/_autosummary/ansys.aedt.core.visualization.post.fields_calculator.FieldsCalculator.export.html
- [4] PyAEDT documentation, PostProcessor3D.export_report_to_csv.
https://aedt.docs.pyansys.com/version/stable/API/visualization/_autosummary/ansys.aedt.core.visualization.post.post_common_3d.PostProcessor3D.export_report_to_csv.html

- [5] PyAEDT documentation, PostProcessor3D.get_solution_data.
https://aedt.docs.pyansys.com/version/stable/API/visualization/_autosummary/ansys.aedt.core.visualization.post.post_common_3d.PostProcessor3D.get_solution_data.html
- [6] PyAEDT documentation, Hfss.export_convergence.
https://aedt.docs.pyansys.com/version/stable/API/_autosummary/ansys.aedt.core.hfss.Hfss.export_convergence.html
- [7] PyAEDT documentation, Hfss.export_mesh_stats.
https://aedt.docs.pyansys.com/version/stable/API/_autosummary/ansys.aedt.core.hfss.Hfss.export_mesh_stats.html
- [8] PyAEDT documentation, Hfss.export_profile.
https://aedt.docs.pyansys.com/version/stable/API/_autosummary/ansys.aedt.core.hfss.Hfss.export_profile.html
- [9] PyAEDT documentation, PostProcessor3D.export_mesh_obj.
https://aedt.docs.pyansys.com/version/stable/API/visualization/_autosummary/ansys.aedt.core.visualization.post.post_common_3d.PostProcessor3D.export_mesh_obj.html
- [10] Ansys HFSS Help, Using the Fields Calculator: eigenmode fields are peak-normalized because no real excitation exists. <https://ansyshelp.ansys.com/public/Views/Secured/Electronics/v252/en/Subsystems/HFSS/Content/ReportsandPostProc/UsingtheFieldsCalculator.htm>
- [11] Ansys HFSS Help, Eigenmode Solutions: unloaded Q and eigenmode restrictions.
<https://ansyshelp.ansys.com/public/Views/Secured/Electronics/v252/en/Subsystems/HFSS/Content/HFSS/EigenmodeSolutions.htm>
- [12] Pydantic documentation, data validation and model serialization. <https://pydantic.dev/docs/validation/latest/get-started/>
- [13] h5py documentation, groups, datasets, and attributes. <https://docs.h5py.org/en/latest/high/group.html> and <https://docs.h5py.org/en/latest/high/dataset.html>
- [14] User repository, Superconducting Microwave Resonators Replication.
<https://github.com/Ahmadh1231/Argonne/tree/main>

Source access date: 23 July 2026.

Appendix G. First working-day checklist

- [] Create the new `qresaudit` repository and select a license.
- [] Create the package structure and `pyproject.toml`.
- [] Implement `Diagnostic`, `FileRecord`, `FieldRecord`, `HFSSRunManifest`, and `ExportConfig`.
- [] Generate and commit JSON schemas from the models.
- [] Implement checksum generation and atomic staging.
- [] Create one synthetic valid bundle and make `qresaudit validate` pass.
- [] Create corrupted copies and confirm each intended validation error.
- [] Implement the PyAEDT session wrapper and `inspect` command.
- [] Select or create minimal driven and eigenmode HFSS fixtures.
- [] Record the installed AEDT and PyAEDT versions before coding vendor adapters.

Stop condition for scope control: Do not begin Q fitting, participation ratios, one-photon normalization, spin coupling, or a graphical interface until both HFSS fixture bundles export and validate end-to-end. The export contract is the foundation for every later module.